Call Me Maybe

Cours 8 — Assemblage final

---

1. L'ordre logique de codage

Chaque etape est testable independamment. Ne passe jamais a l'etape suivante sans avoir valide la precedente.

#	Fichier	Ce qu'on fait	Test possible
1	`src/models.py`	Classes Pydantic — aucune logique	Creer des objets a la main
2	`src/loader.py`	Lire et valider les JSON d'entree	Charger les vrais fichiers
3	`src/vocabulary.py`	Charger le vocab depuis le SDK	Verifier len(vocab) ~150 000
4	`func_tokens`	Pre-calculer les token IDs des noms de fonctions	Afficher les tokens de chaque nom
5	`get_valid_tokens()`	Etat par etat, pas tout d'un coup	Tester chaque etat isole
6	`_update_state()`	Transitions d'etat	Creer un GenContext a la main
7	`Boucle — 1 prompt`	Uniquement 'Greet shrek' en premier	Afficher generated_json
8	`11 prompts`	Tous les prompts, regler les edge cases	Comparer avec le schema attendu
9	`src/__main__.py`	Cablage CLI — seulement quand ca tourne	make run

2. src/models.py

```
"""Modeles Pydantic pour le projet call me maybe."""
from pydantic import BaseModel, Field
class ParameterDef(BaseModel):
    """Definition d'un parametre de fonction."""
    type: str
class FunctionDef(BaseModel):
    """Definition complete d'une fonction disponible."""
    name       : str
    description: str
    parameters : dict[str, ParameterDef]
    returns    : ParameterDef
class TestPrompt(BaseModel):
    """Un prompt de test a traiter."""
    prompt: str
class FunctionCall(BaseModel):
    """Resultat d'un appel de fonction genere."""
    prompt    : str
    name      : str = Field(..., min_length=1)
    parameters: dict
```

3. src/loader.py

```
"""Chargement et validation des fichiers JSON."""
import json
import os
from pydantic import ValidationError
from src.models import FunctionDef, TestPrompt, FunctionCall
def load_function_definitions(path: str) -> list[FunctionDef]:
    """Charge et valide les definitions de fonctions."""
    try:
        with open(path) as f:
            raw = json.load(f)
    except FileNotFoundError:
        raise FileNotFoundError(f'Fichier introuvable : {path}')
    except json.JSONDecodeError as e:
        raise ValueError(f'JSON invalide : {e}')
    functions: list[FunctionDef] = []
    for item in raw:
        try:
            functions.append(FunctionDef(**item))
        except ValidationError as e:
            raise ValueError(f'Definition invalide : {e}')
    return functions
def load_test_prompts(path: str) -> list[TestPrompt]:
    """Charge et valide les prompts de test."""
    try:
        with open(path) as f:
            raw = json.load(f)
    except FileNotFoundError:
        raise FileNotFoundError(f'Fichier introuvable : {path}')
    except json.JSONDecodeError as e:
        raise ValueError(f'JSON invalide : {e}')
    prompts: list[TestPrompt] = []
    for item in raw:
        try:
            prompts.append(TestPrompt(**item))
        except ValidationError as e:
            raise ValueError(f'Schema invalide : {e}')
    return prompts
def save_results(results: list[FunctionCall], path: str) -> None:
    """Sauvegarde les resultats dans un fichier JSON."""
    os.makedirs(os.path.dirname(path), exist_ok=True)
    with open(path, 'w') as f:
        json.dump([r.model_dump() for r in results], f, indent=2)
```

4. src/vocabulary.py

```
"""Chargement du vocabulaire depuis le SDK."""
import json
from llm_sdk import Small_LLM_Model
from src.models import FunctionDef
def load_vocabulary(model: Small_LLM_Model) -> dict[str, int]:
    """Charge le vocabulaire token -> ID depuis le tokenizer."""
    path = model.get_path_to_tokenizer_file()
    with open(path) as f:
        data = json.load(f)
    try:
        return data["model"]["vocab"]
    except KeyError as e:
        raise ValueError(f'Structure tokenizer inattendue : {e}')
def build_id_to_token(vocab: dict[str, int]) -> dict[int, str]:
    """Construit la table inverse ID -> token."""
    return {v: k for k, v in vocab.items()}
def precompute_function_tokens(
    functions: list[FunctionDef],
    model: Small_LLM_Model,
) -> dict[str, list[int]]:
    """Pre-calcule les token IDs de chaque nom de fonction."""
    result: dict[str, list[int]] = {}
    for func in functions:
        result[func.name] = model.encode(func.name)[0].tolist()
    return result
```

5. Valider les func_tokens avant tout le reste

Avant d'ecrire le generator, execute ce script de verification. Si les tokens sont faux ici, get_valid_tokens() ne convergera jamais.

```
# test_tokens.py — a lancer directement : uv run python test_tokens.py
from llm_sdk import Small_LLM_Model
from src.vocabulary import load_vocabulary, precompute_function_tokens
from src.loader import load_function_definitions
model     = Small_LLM_Model()
vocab     = load_vocabulary(model)
functions = load_function_definitions(
    'data/input/functions_definition.json'
)
func_tokens = precompute_function_tokens(functions, model)
print(f'Vocabulaire : {len(vocab)} tokens')
print()
for name, ids in func_tokens.items():
    tokens = [model.decode([i]) for i in ids]
    print(f'{name}')
    print(f'  IDs    : {ids}')
    print(f'  Tokens : {tokens}')
    print()
# Resultat attendu :
# fn_add_numbers
#   Tokens : ['fn', '_add', '_numbers']
# fn_greet
#   Tokens : ['fn', '_greet']
# fn_reverse_string
#   Tokens : ['fn', '_reverse', '_string']
# fn_get_square_root
#   Tokens : ['fn', '_get', '_square', '_root']
# fn_substitute_string_with_regex
#   Tokens : ['fn', '_substitute', '_string', '_with', '_regex']
```

6. src/generator.py — structure generale

```
# Ordre d'implementation dans generator.py :
# 1. JsonState (enum) + GenContext (dataclass)
# 2. _get_number_tokens() — tokens valides pour un nombre
# 3. get_valid_tokens()   — un etat a la fois, tester au fur et a mesure
# 4. _update_state()      — transitions, tester avec GenContext manuel
# 5. generate_function_call() — boucle complete, tester sur 'Greet shrek'
# 6. generate_all()       — tous les prompts + sauvegarde
# Astuce : ajouter ce mode debug dans la boucle
DEBUG = True   # passer a False pour la production
while ctx.state != JsonState.END:
    logits    = model.get_logits_from_input_ids(input_ids)
    logits_np = np.array(logits)
    valid_ids = get_valid_tokens(ctx, vocab, model)
    if not valid_ids:
        raise ValueError(f'Aucun token valide en etat {ctx.state}')
    mask = np.full(len(logits_np), -np.inf)
    mask[valid_ids] = 0.0
    logits_np = logits_np + mask
    next_id    = int(np.argmax(logits_np))
    next_token = id_to_token.get(next_id, '')
    if DEBUG:
        print(f'  {ctx.state.name:15} | {repr(next_token):20} | {next_id}')
    input_ids.append(next_id)
    generated_json += next_token
    _update_state(ctx, next_id, next_token)
```

7. Sortie attendue pour les 11 prompts

```
# function_calling_results.json
[
  {
    "prompt": "What is the sum of 2 and 3?",
    "name": "fn_add_numbers",
    "parameters": {"a": 2.0, "b": 3.0}
  },
  {
    "prompt": "Greet shrek",
    "name": "fn_greet",
    "parameters": {"name": "shrek"}
  },
  {
    "prompt": "Replace all vowels in 'Programming is fun' with asterisks",
    "name": "fn_substitute_string_with_regex",
    "parameters": {
      "source_string": "Programming is fun",
      "regex": "[aeiou]",
      "replacement": "*"
    }
  },
  ...
]
```

8. Problemes courants et solutions

#	Symptome	Cause probable	Solution
1	Boucle infinie sur FUNC_NAME	func_tokens mal calcules ou comparaison token_ids[:n] incorrecte	Afficher name_so_far a chaque tour, verifier func_tokens
2	ValueError : aucun token valide	get_valid_tokens() retourne [] — etat incoherent	Afficher ctx.state et ctx.chosen_func avant l'appel
3	JSONDecodeError apres la boucle	Accolade manquante ou guillemet non ferme dans generated_json	Afficher generated_json complet avant json.loads()
4	Parametre de mauvais type (string au lieu de number)	La valeur numerique a ete generee entre guillemets	Verifier que PARAM_VALUE pour number ne passe pas par IN_STRING

#	Symptome	Cause probable	Solution
5	fn_substitute mal identifiee	Le nom est long — verifier que tous ses tokens sont dans le vocab	Afficher func_tokens['fn_substitute_string_with_regex']
6	Crash sur model.encode()	encode() retourne un tenseur 2D — oubli de [0].tolist()	Toujours faire model.encode(text)[0].tolist()

9. Le README.md — sections obligatoires

Le sujet impose ces sections specifiques en plus des sections standard :

````
# Call Me Maybe
*This project has been created as part of the 42 curriculum by <login>.*
## Description
Programme de function calling : traduit des phrases en langage naturel
en appels de fonctions JSON structures via constrained decoding sur Qwen3-0.6B.
## Instructions
```bash
cp -r /chemin/llm_sdk ./llm_sdk
uv sync
make run
```
## Algorithm explanation
Description de l'approche constrained decoding...
## Design decisions
Choix cles de l'implementation...
## Performance analysis
Precision, vitesse, fiabilite...
## Challenges faced
Difficultes rencontrees et solutions...
## Testing strategy
Comment la solution a ete validee...
## Example usage
```bash
uv run python -m src \
  --functions_definition data/input/functions_definition.json \
  --input data/input/function_calling_tests.json \
  --output data/output/function_calling_results.json
```
## Resources
- Liens vers docs, articles, tutoriels
- Description de l'utilisation de l'IA dans le projet
````

Exercice final

Q1. Pourquoi encode-t-on prompt + FIXED_PREFIX ensemble plutot que d'encoder le prompt seul puis d'ajouter les IDs de FIXED_PREFIX separement ?

Q2. Apres AFTER_NAME, on injecte FIXED_SUFFIX sans passer par le LLM. Pourquoi est-ce correct ?

Q3. Le modele genere le token '3}' pendant PARAM_VALUE pour un nombre. Trace le chemin complet dans _update_state().

Reponses

Q 1	Le LLM est un modele de contexte : la facon dont le prompt et la structure fixe sont joints influence les logits. Encoder les deux ensemble garantit que le modele voit la transition naturelle entre le prompt et le debut du JSON. Encoder separement et concatener les IDs produirait un contexte different que le modele n'a pas appris a traiter.
Q 2	La structure fixe '", "parameters": {"' est deterministe — elle ne depend pas du prompt ni de la fonction choisie. Elle est toujours identique. Passer par le LLM serait inutile et risque : le constrained decoding ne laisserait de toute facon qu'un seul token valide a chaque position. L'injecter directement est plus rapide et plus sur.
Q 3	Le token '3}' est genere pendant PARAM_VALUE. Dans _update_state(), on entre dans le bloc PARAM_VALUE. ptype = 'number' donc on ne va pas en IN_STRING. On fait current_value += '3}'. next_token.strip() = '3}' — strip() ne retire que les espaces, donc '3}' ne matche pas ',' ni '}'. ATTENTION : c'est un bug potentiel ! Il faut verifier si '}' est CONTENU dans next_token, pas si next_token EST '}'. Correction : if '}' in next_token.strip() or ',' in next_token.strip().

A retenir

- Coder dans l'ordre : models -> loader -> vocabulary -> func_tokens -> get_valid_tokens -> boucle.
- Tester func_tokens avant d'ecrire le generator — c'est la base de tout.
- Commencer par 'Greet shrek' (1 param, nom court) avant les 11 prompts.
- Ajouter DEBUG = True dans la boucle pour voir chaque etat et token.
- Le README doit inclure : algorithm explanation, design decisions, performance analysis, challenges faced, testing strategy, example usage.

Tu es pret a coder !

Tu as maintenant tous les concepts, tous les outils, et tous les fichiers du projet. Suis l'ordre des etapes, teste a chaque niveau, et n'hesite pas a revenir aux PDFs des cours precedents si tu bloques.